

# Thinning Thinnings: Safe and Efficient Binders

APRIL GONÇALVES, University of Strathclyde, UK

WEN KOKKE, Well-Typed, UK

To be written last...

CCS Concepts: • **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Additional Key Words and Phrases: **TODO: Set keywords**

## ACM Reference Format:

April Gonçalves and Wen Kokke. 2018. Thinning Thinnings: Safe and Efficient Binders. *J. ACM* 37, 4, Article 111 (August 2018), 3 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

- Define the  $ST\lambda C$  using *safe* combinators.
- Explain the efficient issues with safe combinators.

**type data**  $Nat = Z \mid S\ Nat$

**data**  $Ix\ (n :: Nat)\ \text{where}$

$FZ :: Ix\ (S\ n)$

$FS :: Ix\ n \rightarrow Ix\ (S\ n)$

**data**  $Ty$

$= Bot$

$\mid Ty \rightarrow Ty$

**data**  $Tm_{\uparrow}\ n$

$= Ann\ (Tm_{\downarrow}\ n)\ Ty$

$\mid Var\ (Ix\ n)$

$\mid App\ (Tm_{\uparrow}\ n)\ (Tm_{\downarrow}\ n)$

**data**  $Tm_{\downarrow}\ n$

$= Inf\ (Tm_{\uparrow}\ n)$

$\mid Lam\ (Tm_{\downarrow}\ (S\ n))$

---

Authors' Contact Information: April Gonçalves, [april@cyberglot.me](mailto:april@cyberglot.me), University of Strathclyde, Glasgow, UK; Wen Kokke, [me@wen.works](mailto:me@wen.works), Well-Typed, Glasgow, UK.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

```

data Env (n :: Nat) (a :: Type) where
  Nil :: Env Z a
  (:>) :: Env n a → a → Env (S n) a

  (!! ) :: Env n a → Ix n → a
  γ :> a !! FZ = a
  γ :> a !! FS i = γ !! i

  check :: Env n Ty → Tm↓ n → Ty → Bool
  check γ (Inf e) τ = infer γ e ≡ Just τ
  check γ (Lam e) (τ :→ τ') = check (γ :> τ) e τ'

  infer :: Env n Ty → Tm↑ n → Maybe Ty
  infer γ = λcase
    Ann e τ | check γ e τ → Just τ
    Var i → Just (γ !! i)
    App e e' | Just (τ :→ τ') ← infer γ e, check γ e' τ → Just τ'
    — → Nothing

  idTm :: Tm↑ n
  idTm = Ann (Lam (Inf (Var FZ))) (Bot :→ Bot)

```

□ Add an example that requires thinnings and composition

## 2 The Fast And The Faithful

- Explain the technique to control the type representation through lies.
- Define the *fast and faithful* combinators.
- Give special attention to thinning composition using PDEP.
- Explain the QuickCheck test suite.
- Explain why we're not proving equivalence.

```

data (m :: Nat) ≥ (n :: Nat) where
  KeepAll :: m ≥ m
  KeepOne :: m ≥ n → S m ≥ S n
  DropOne :: m ≥ n → S m ≥ n

```

```

data Th f m = Th (m ≥ n) (f n)

```

```

data Tm↑ (n :: Nat) where
  Ann :: Tm↓ n → Ty → Tm↑ n
  Var :: Tm↑ Z
  App :: Th Tm↑ n → Th Tm↓ n → Tm↑ n

```

```

data Tm↓ (n :: Nat) where
  Inf :: Tm↑ n → Tm↓ n
  Lam :: Tm↓ (S n) → Tm↓ n

```

### **3 Experimental Evaluation**

- Explain the experimental evaluation.

### **References**

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009